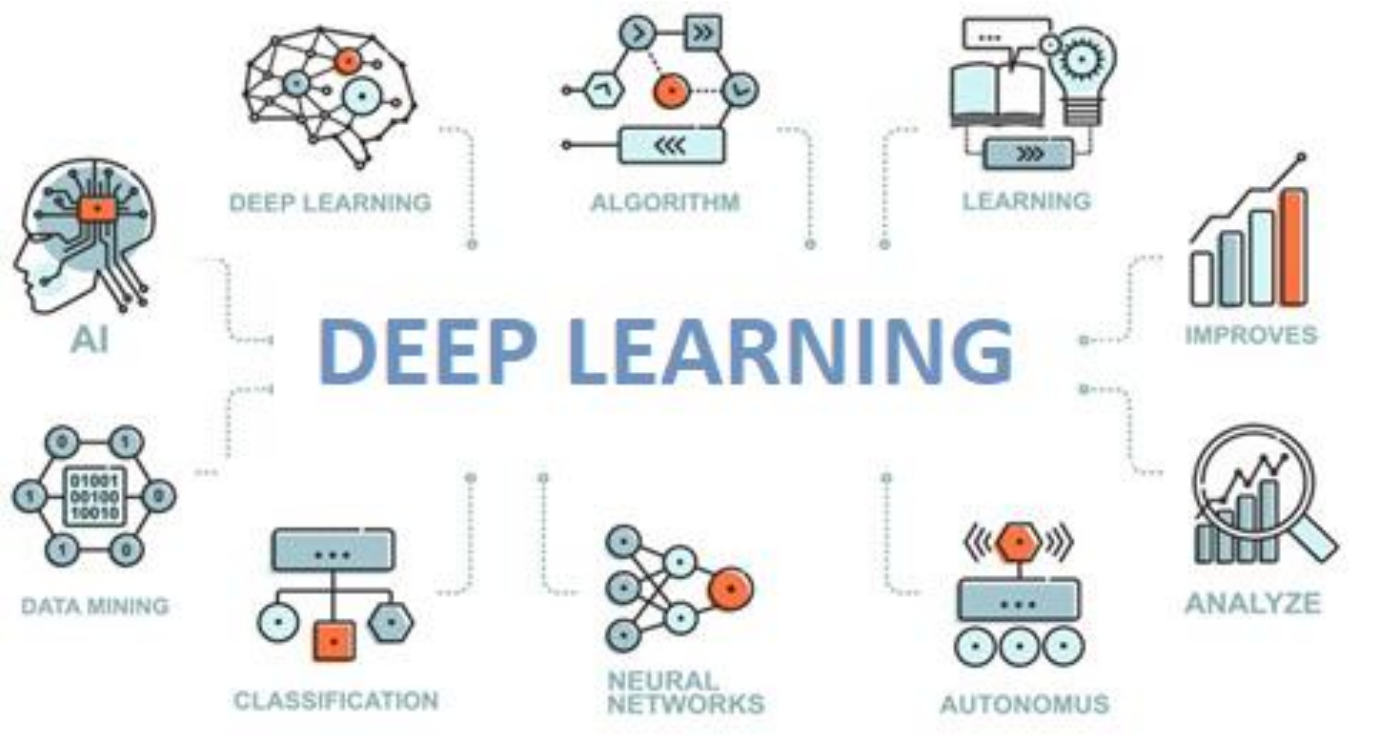
Perceptron



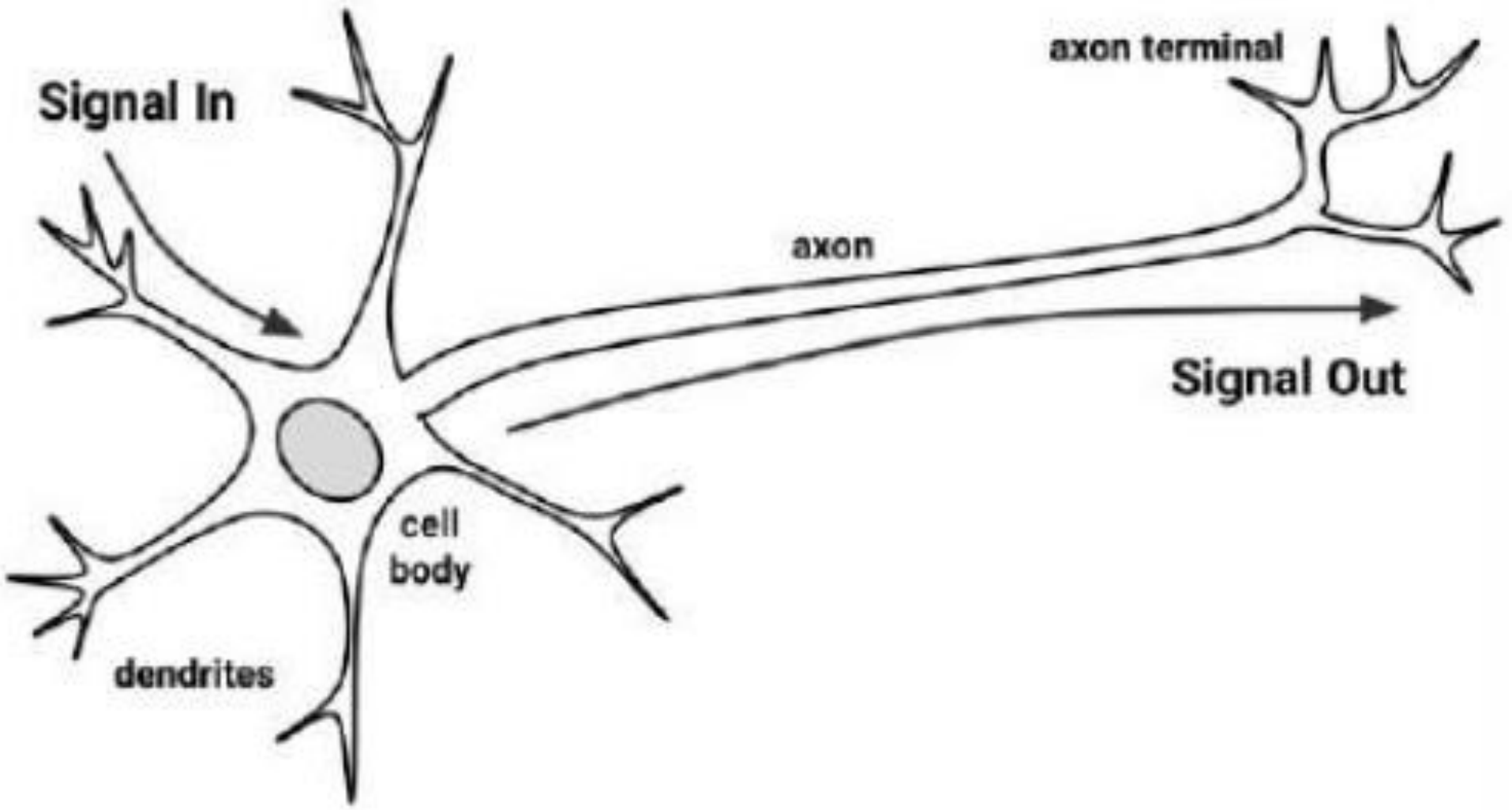
Prepared by: Dr / Doaa Gamal

Assistant professor at Faculty of Engineering, Suez Canal University
(Doaa_gamal@eng.suez.edu.eg)

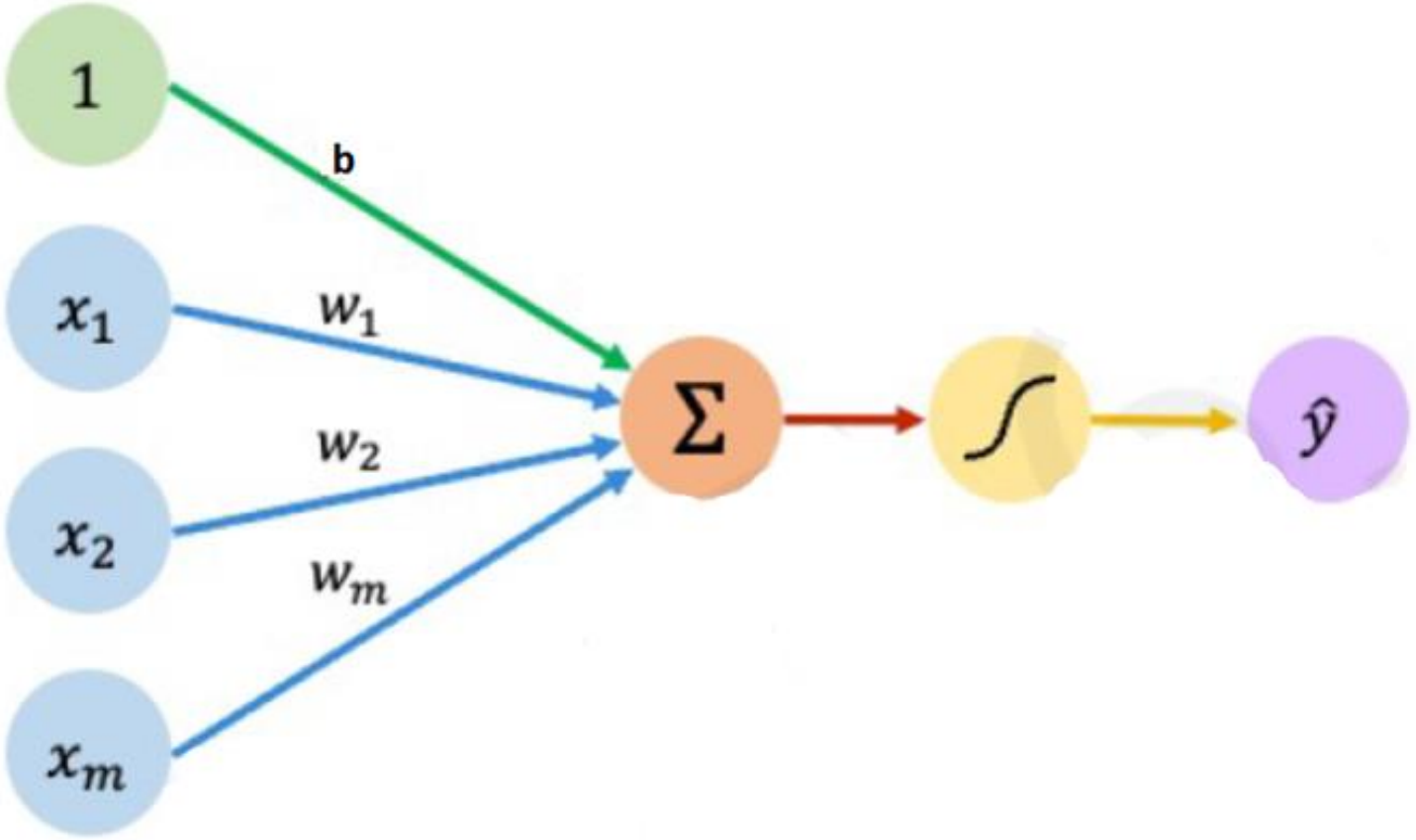
Lecture outline

- Last time we talked about:
  - AI & ML & DL
  - Different types of problems
  - Why deep learning now?
- Today we are going to talk about:
  - perceptron (the basic building block of ANN)
  - Examples (AND gate , XOR gate)

Neural networks



Artificial neural network (ANN)



Linear combination of inputs

Output

$$\hat{y} = g\left(\mathbf{b} + \sum_{i=1}^{m} x_i\, w_i\right)$$

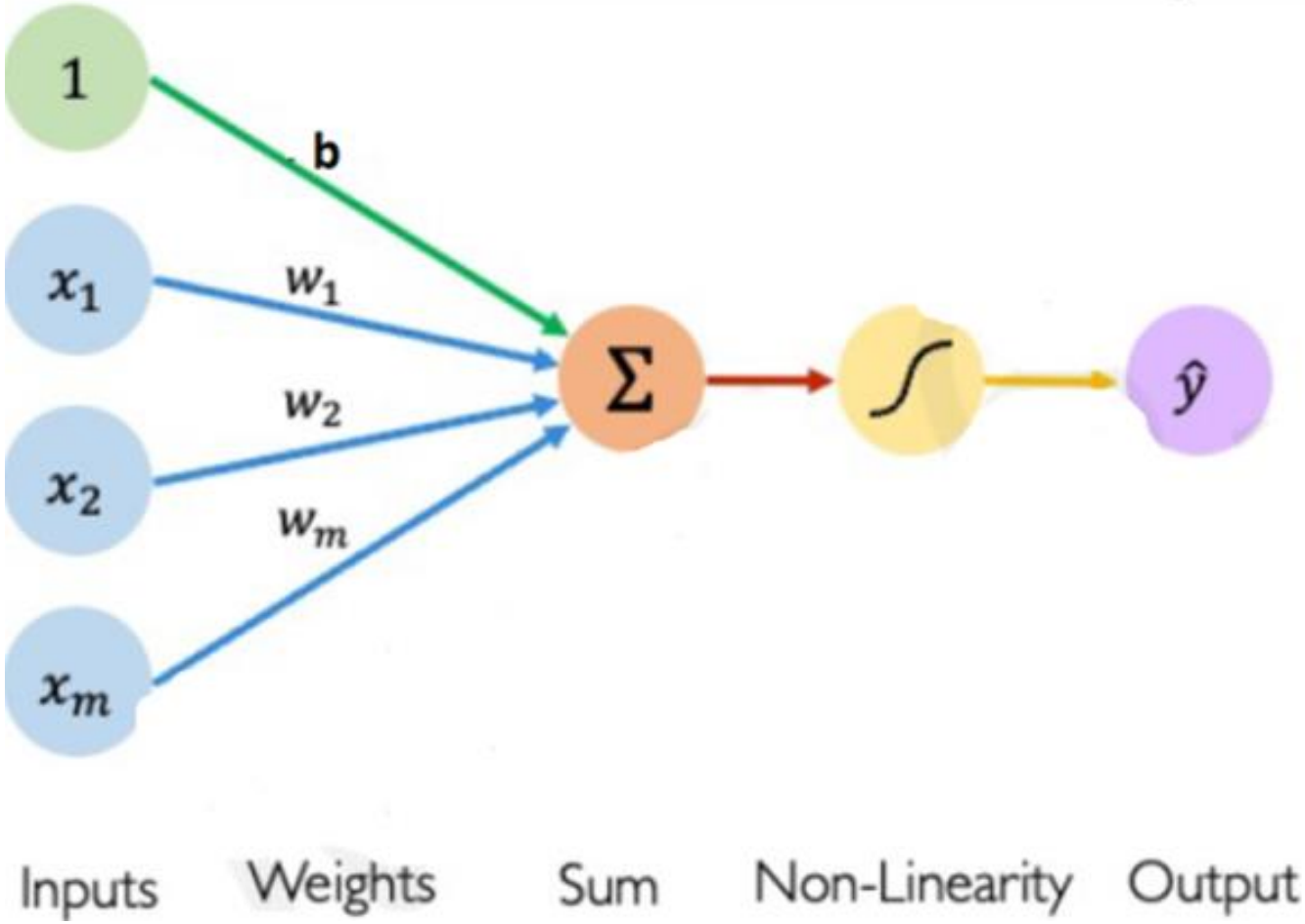
Non-linear activation function

Bias

Inputs | Weights | Sum | Non-Linearity | Output

Perceptron

Perceptron is the basic building block of an ANN



Output

Linear combination of inputs

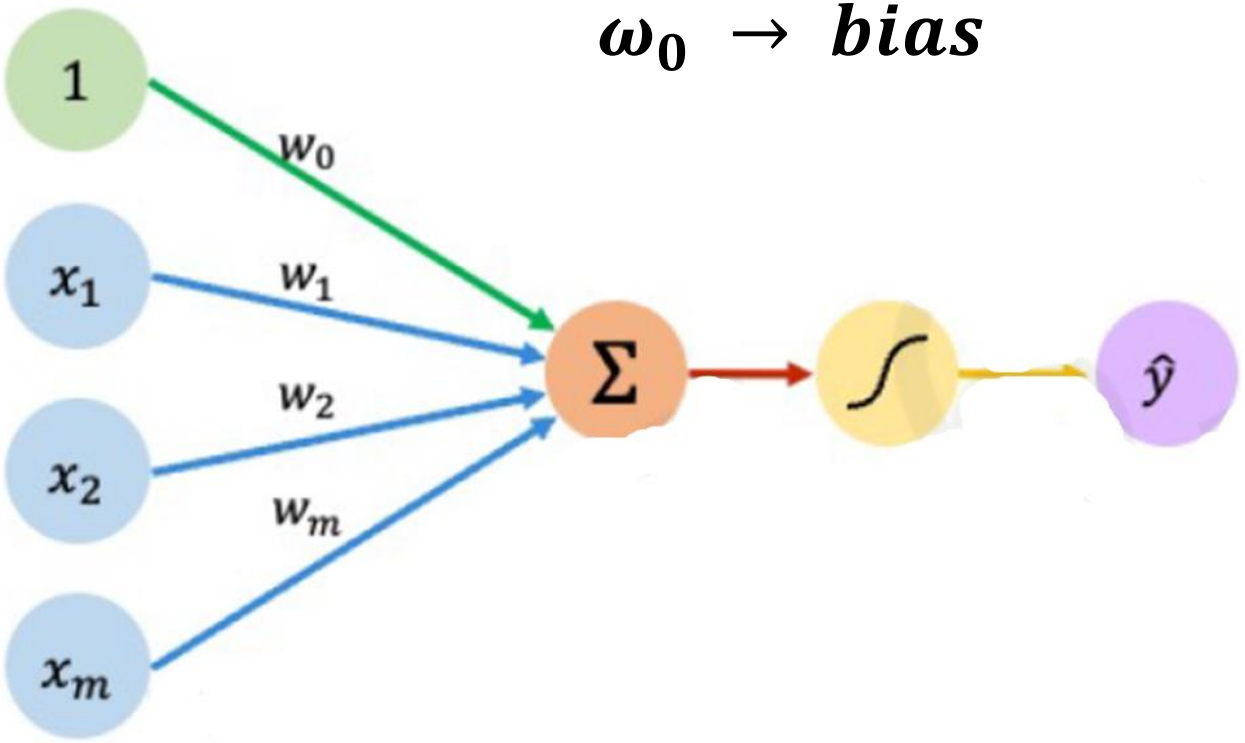
$$\hat{y} = g\left(b + \sum_{i=1}^{m} x_i w_i\right)$$

$$\hat{y} = g(\boldsymbol{W}\boldsymbol{X} + b)$$

where: $\boldsymbol{X} = \begin{bmatrix} x_1 \\ \vdots \\ x_m \end{bmatrix}$ and $\boldsymbol{W} = \begin{bmatrix} w_1 \\ \vdots \\ w_m \end{bmatrix}^T$

Perceptron

$\omega_0 \rightarrow bias$



Inputs Weights Sum Non-Linearity Output

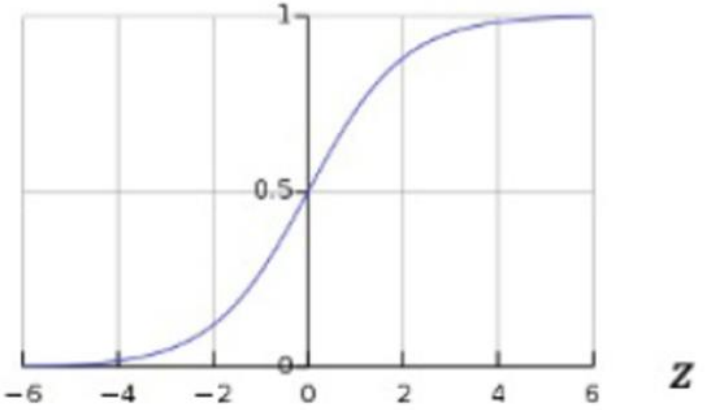
Activation Functions

$$\hat{y} = g(\mathbf{W}\mathbf{X} + \mathbf{b})$$

$$\hat{y} = g(z), \quad z = \mathbf{W}\mathbf{X} + \mathbf{b}$$

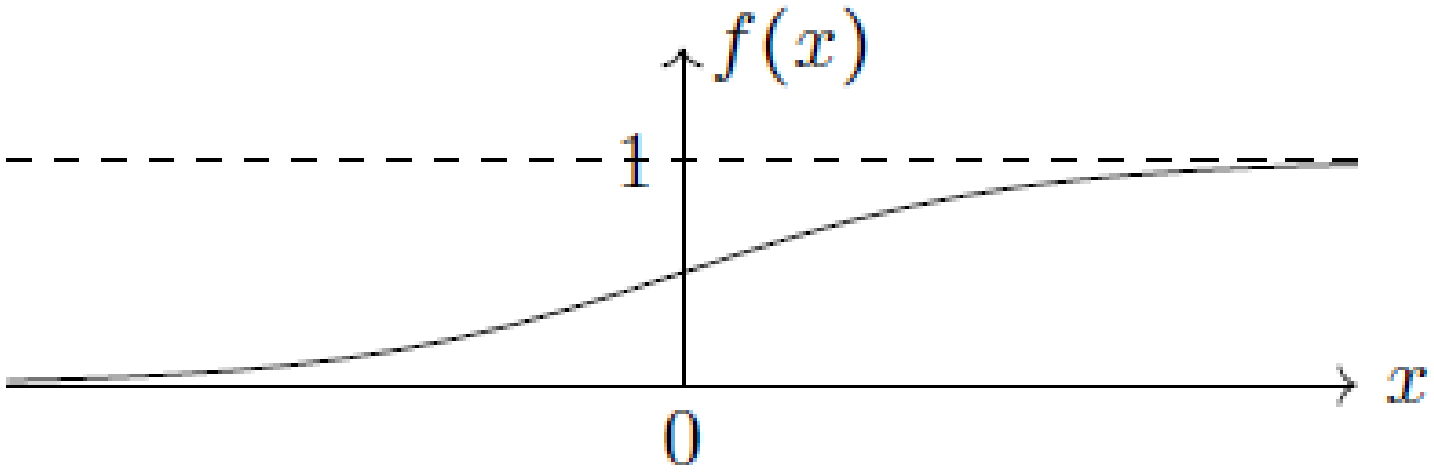
- Example: sigmoid function

$$g(z) = \sigma(z) = \frac{1}{1 + e^{-z}}$$



Activation function

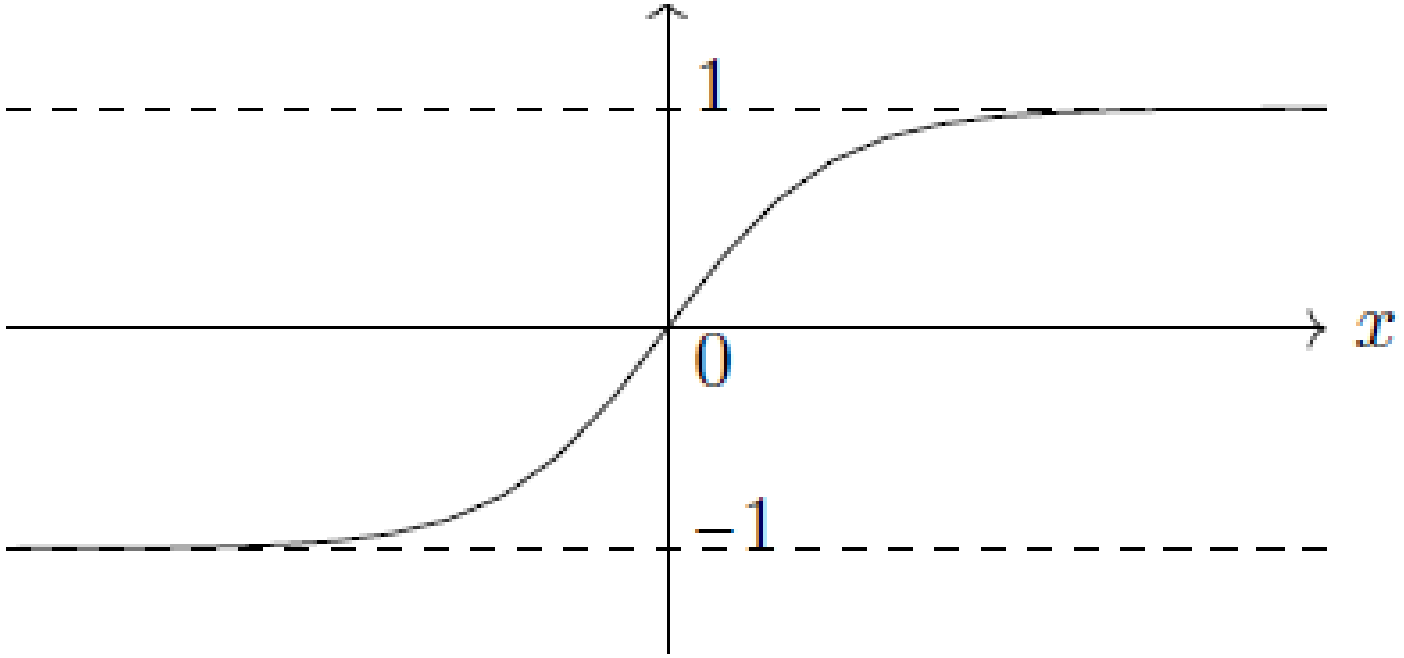
Sigmoid activation function



$$f(x) = \frac{1}{1 + e^{-x}}$$

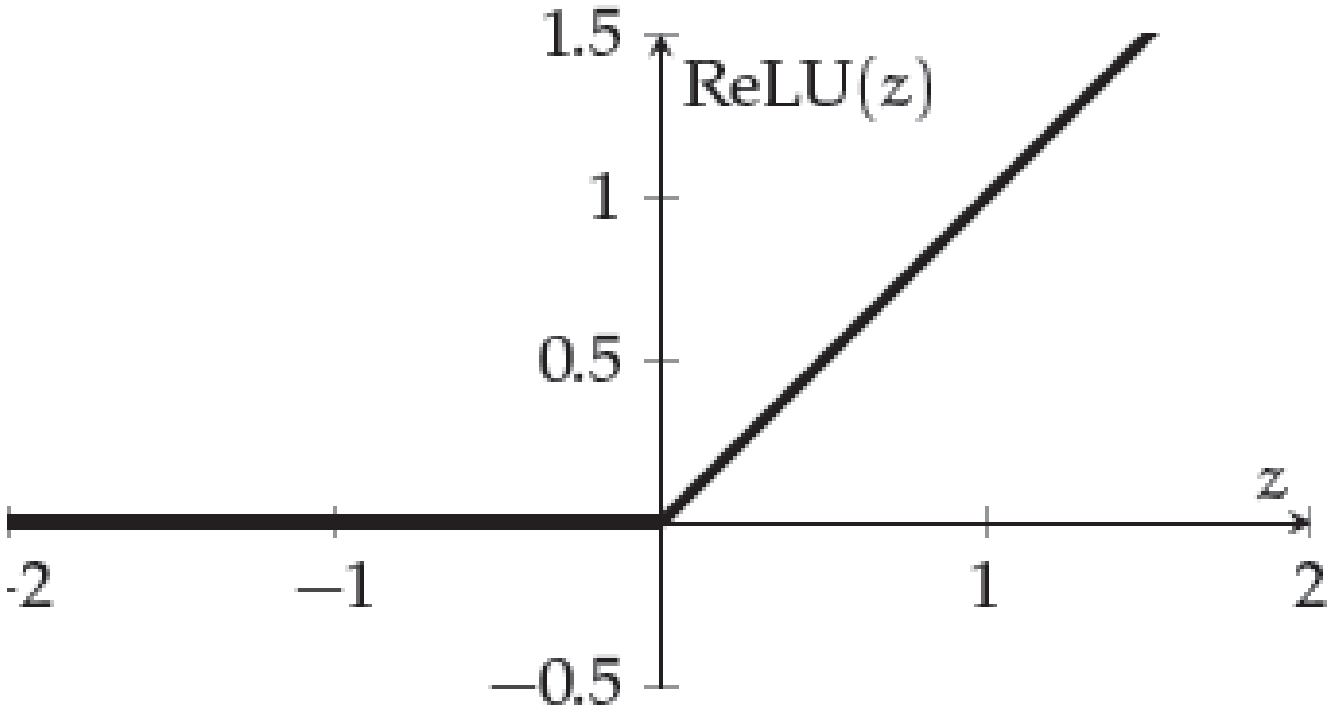
Activation function

Hyperbolic tangential activation function



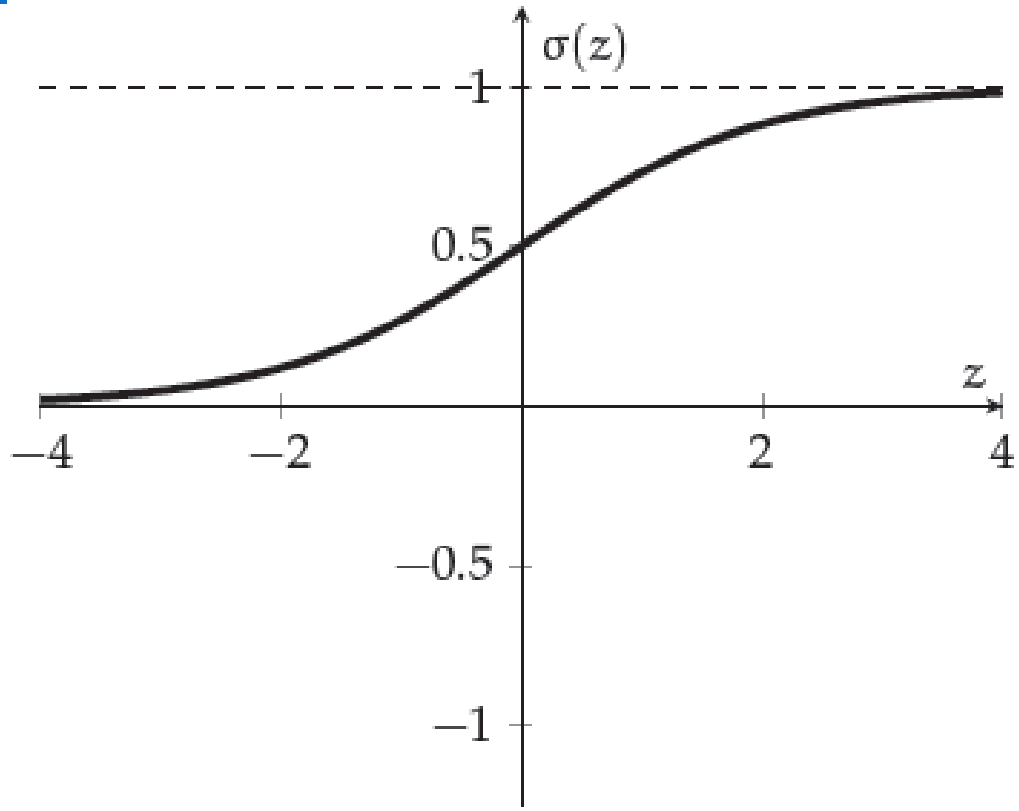
$$f(x) = \frac{e^{x} - e^{-x}}{e^{x} + e^{-x}}.$$

Activation function



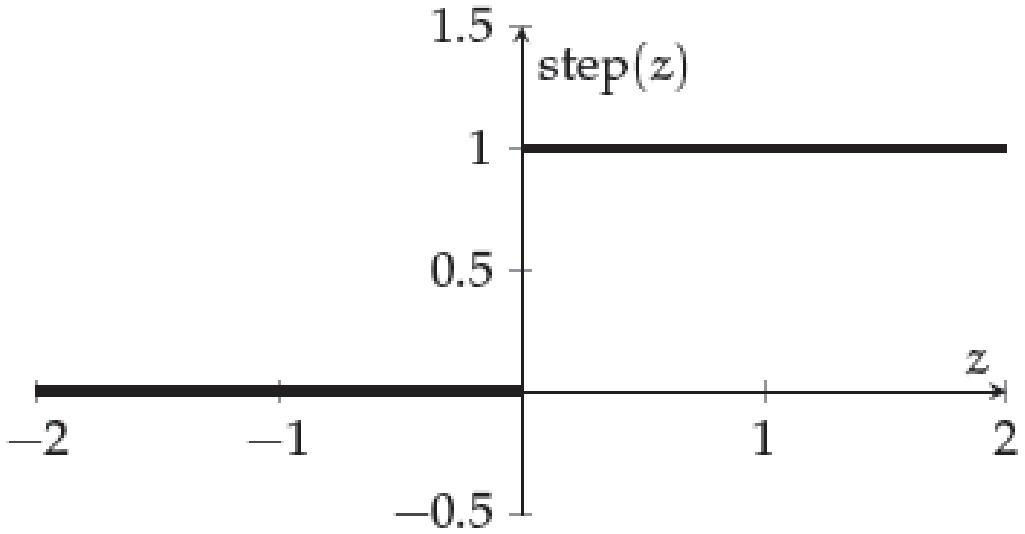
$$f(z) = max\,(0, z)$$

Activation function



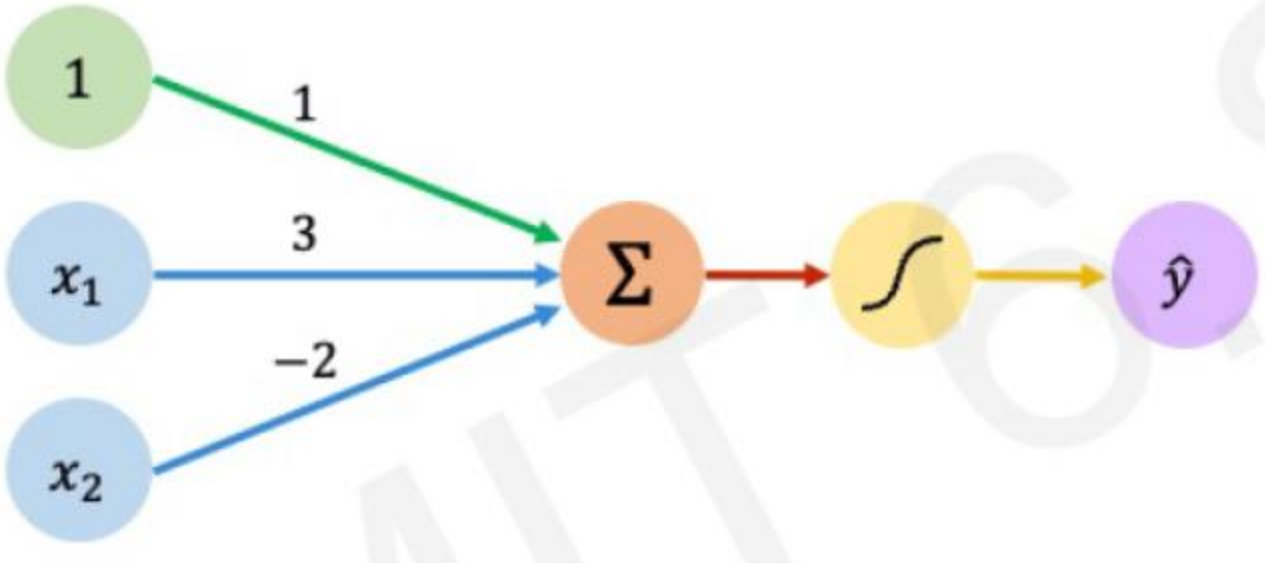
$$\text{softmax}(z) = \begin{bmatrix} \exp(z_1)/\sum_i \exp(z_i) \\ \vdots \\ \exp(z_n)/\sum_i \exp(z_i) \end{bmatrix}$$

Activation function



$$\text{step}(z) = \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{otherwise} \end{cases}$$

The perceptron (example)



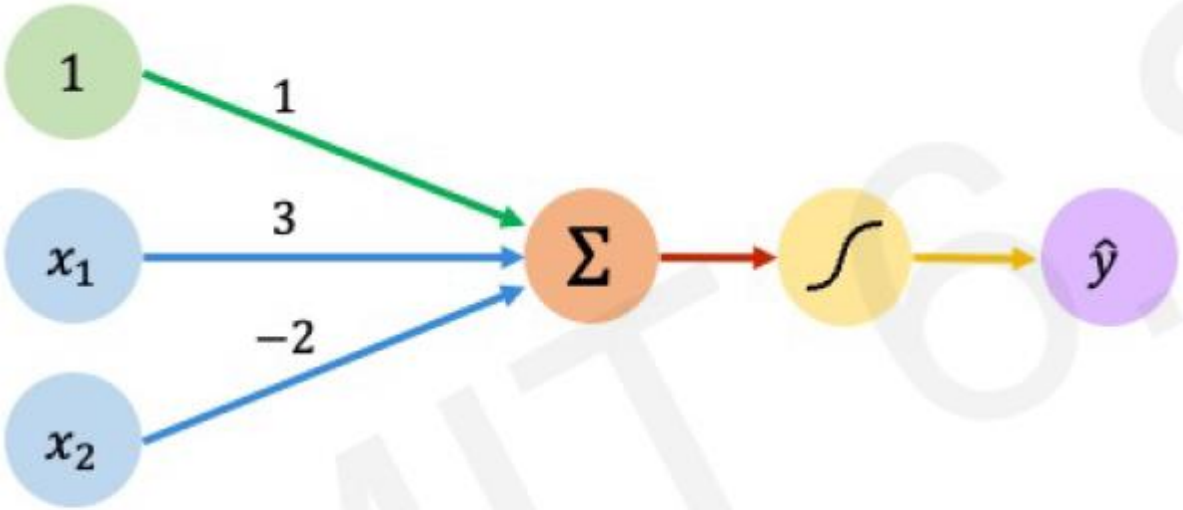
We have: $\mathbf{b} = 1$ and $\boldsymbol{W} = \begin{bmatrix} 3 & -2 \end{bmatrix}$

$$\hat{y} = g(\boldsymbol{W}\boldsymbol{X} + \mathbf{b})$$

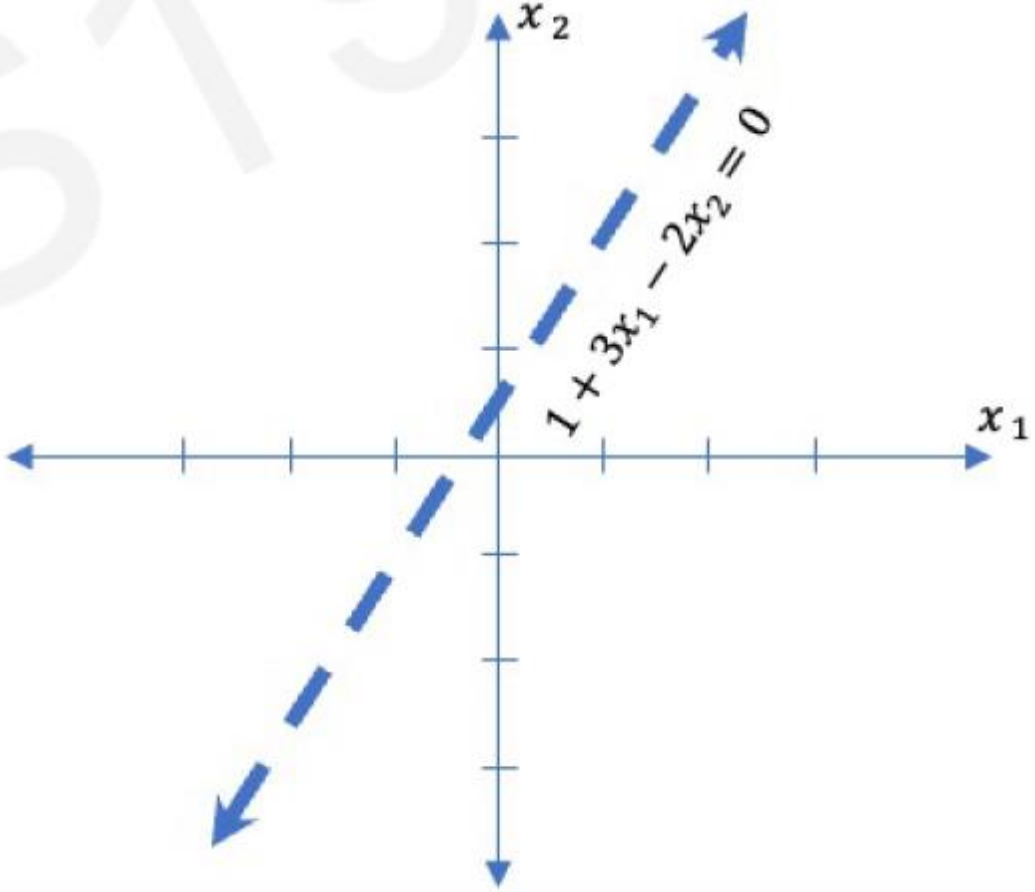
$$\hat{y} = g(1 + 3x_1 - 2x_2)$$

This is just a line in 2D!

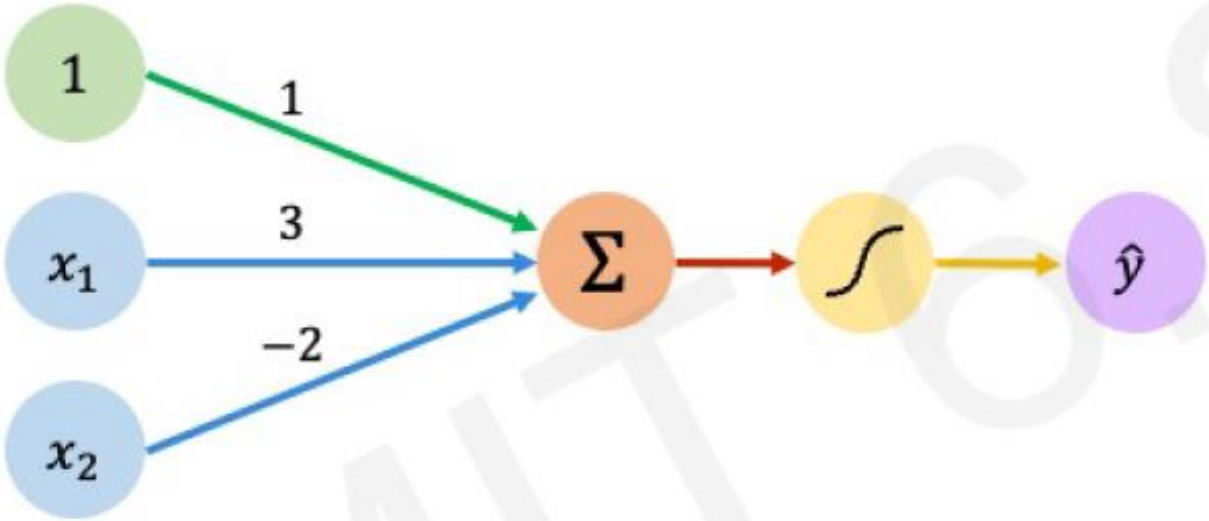
The perceptron (example)



$$\hat{y} = g(1 + 3x_1 - 2x_2)$$



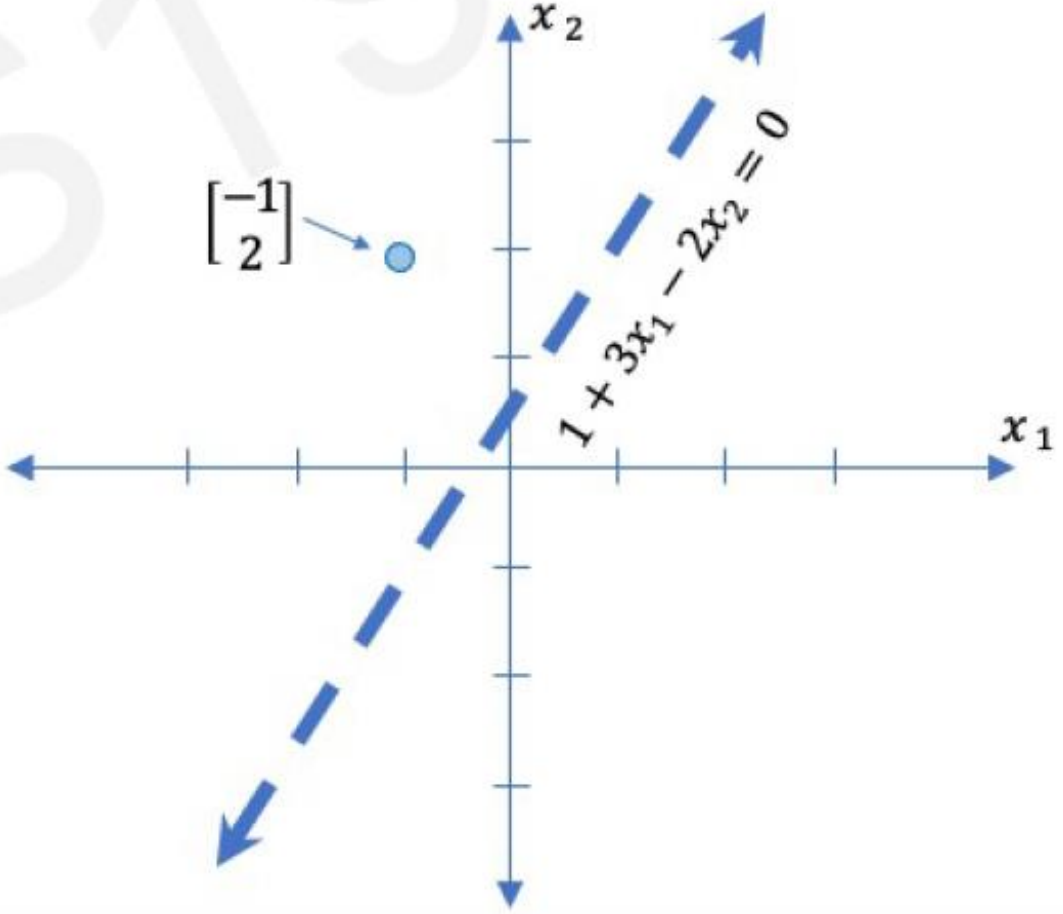
The perceptron (example)



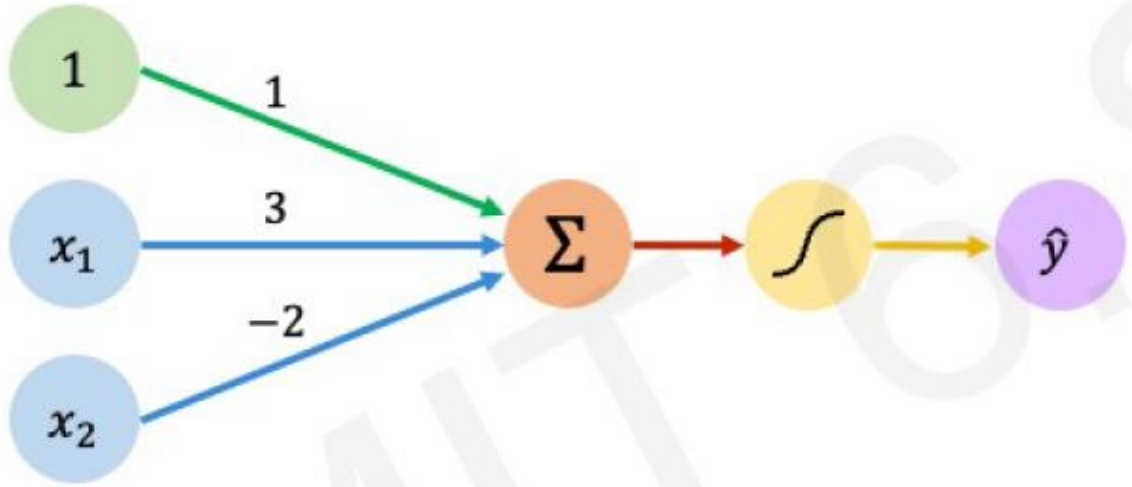
Assume we have input: $\boldsymbol{X} = \begin{bmatrix} -1 \\ 2 \end{bmatrix}$

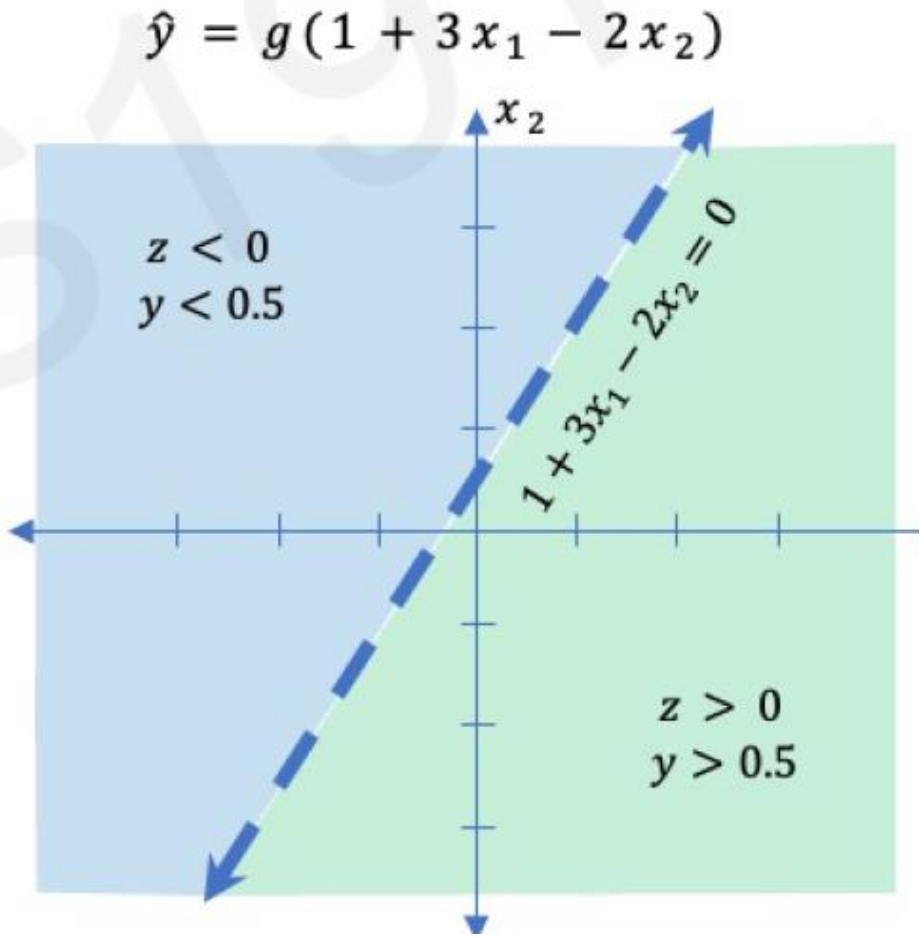
$$\hat{y} = g\left(1 + (3 * -1) - (2 * 2)\right)$$
$$= g(-6) \approx 0.002$$

$$\hat{y} = g(1 + 3x_1 - 2x_2)$$



The perceptron (example)





Example (AND operation)

Table 1: The AND Logical Operation (Binary)

x_1	x_2	$y = x_1$ AND x_2
0	0	0
0	1	0
1	0	0
1	1	1

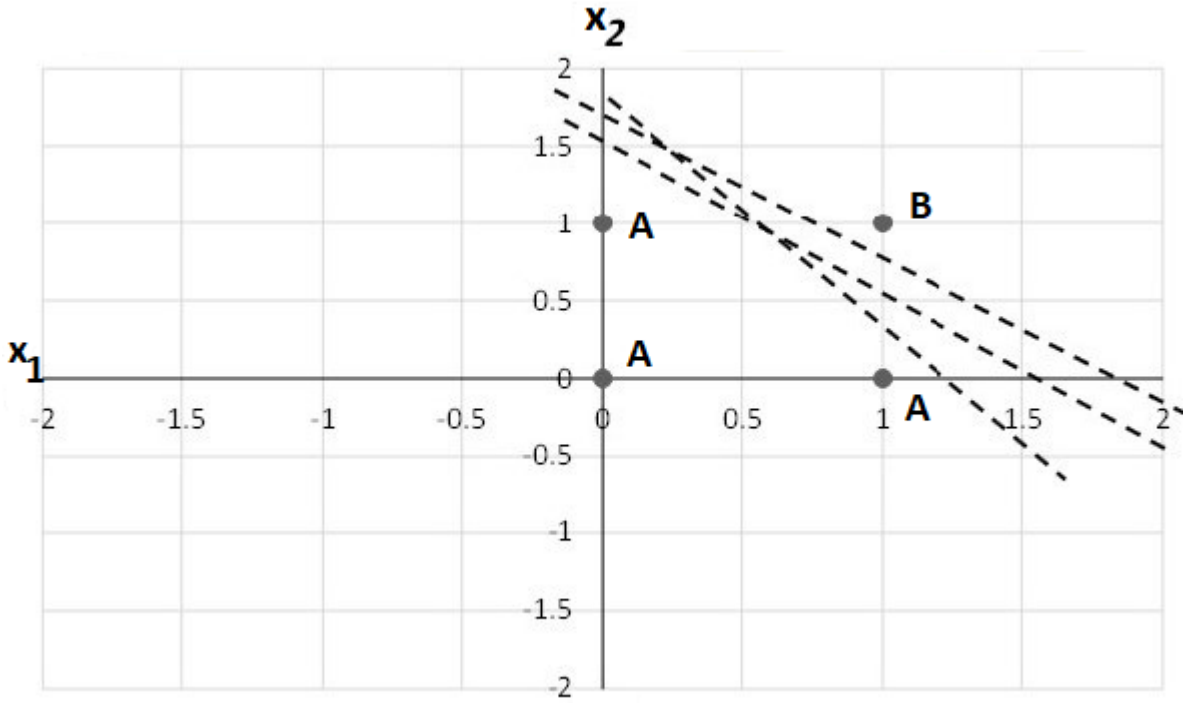
Example (AND operation)

Table 1: The AND Logical Operation (Binary)

x_1	x_2	$y = x_1$ AND x_2
0	0	0 (class A)
0	1	0 (class A)
1	0	0 (class A)
1	1	1 (class B)

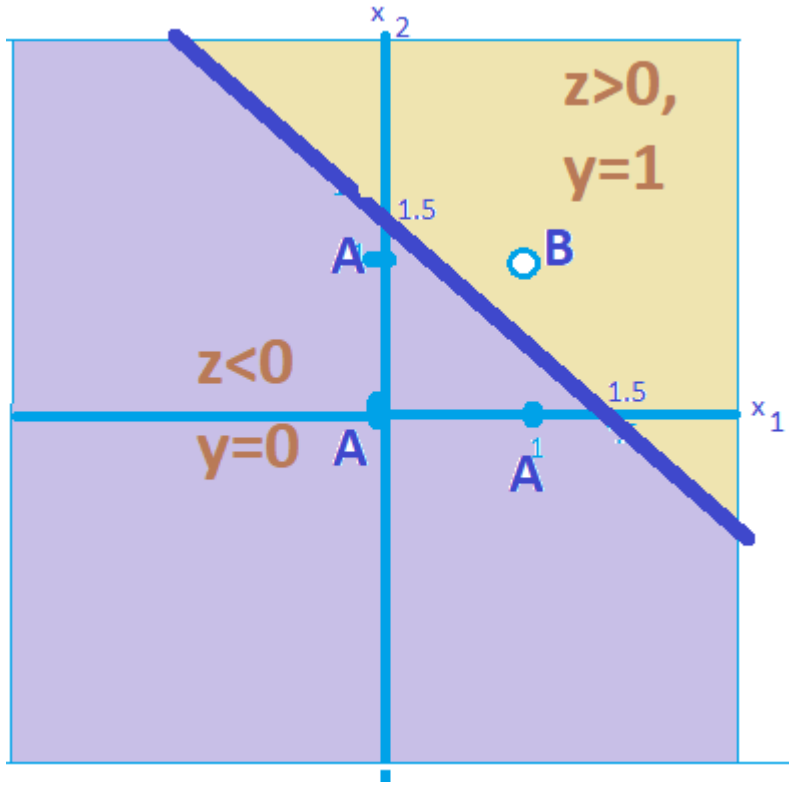
Decision boundary equation

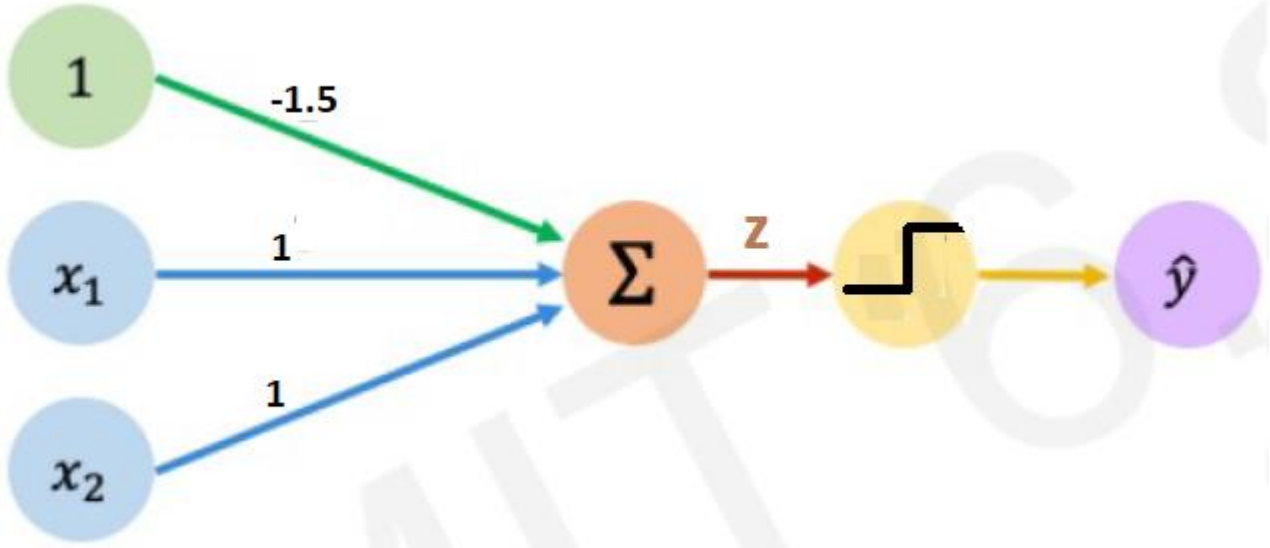
$$x_1 + x_2 - 1.5 = 0$$



Example (AND operation)

$$z = x_1 + x_2 - 1.5$$

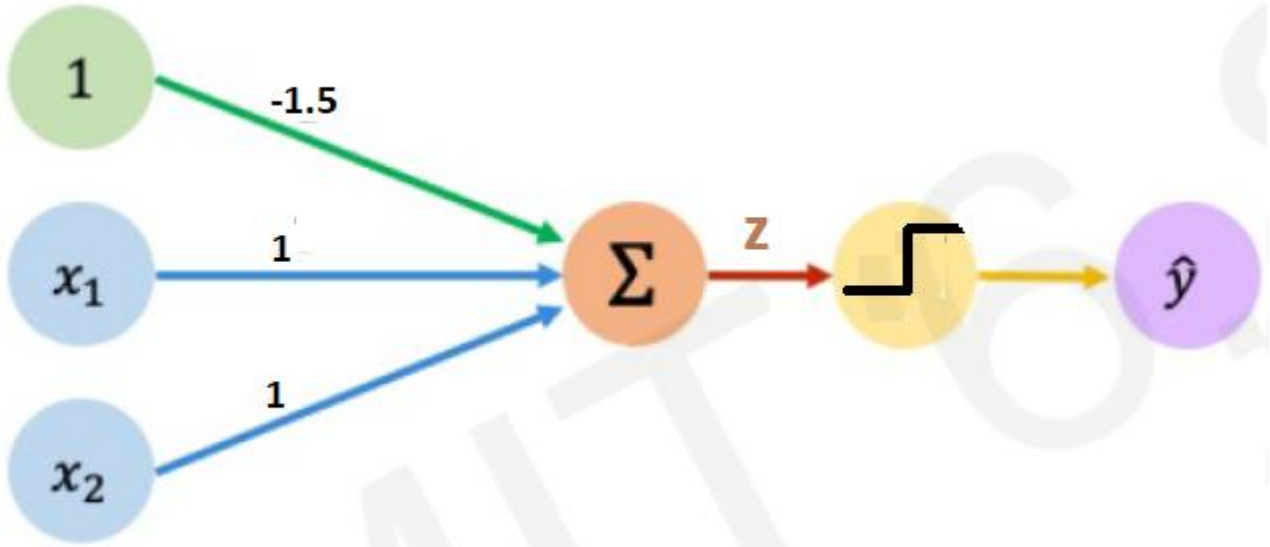




Example (AND operation, test)

$$z = x_1 + x_2 - 1.5$$

x_1	x_2	z	$\hat{y}$	target
0	0	-1.5	0	0
0	1	-0.5	0	0
1	0	-0.5	0	0
1	1	0.5	1	1



Accuracy=100%

Example (OR operation)

Design a neural network that acts as an OR gate with 100% accuracy. Then Check the accuracy (calculate the estimated output for each input)

x_1	x_2	target
0	0	0
0	1	1
1	0	1
1	1	1

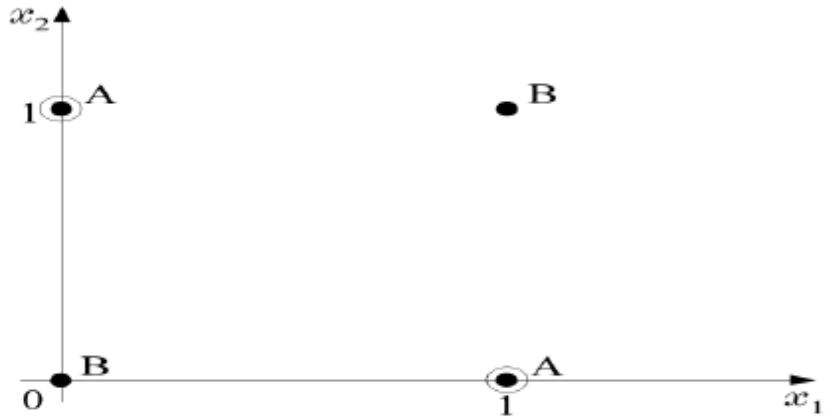
Assignment

Example (XOR)

Design a neural network that acts as an XOR gate with 100% accuracy. Then Check the accuracy (calculate the estimated output for each input)

- The XOR problem

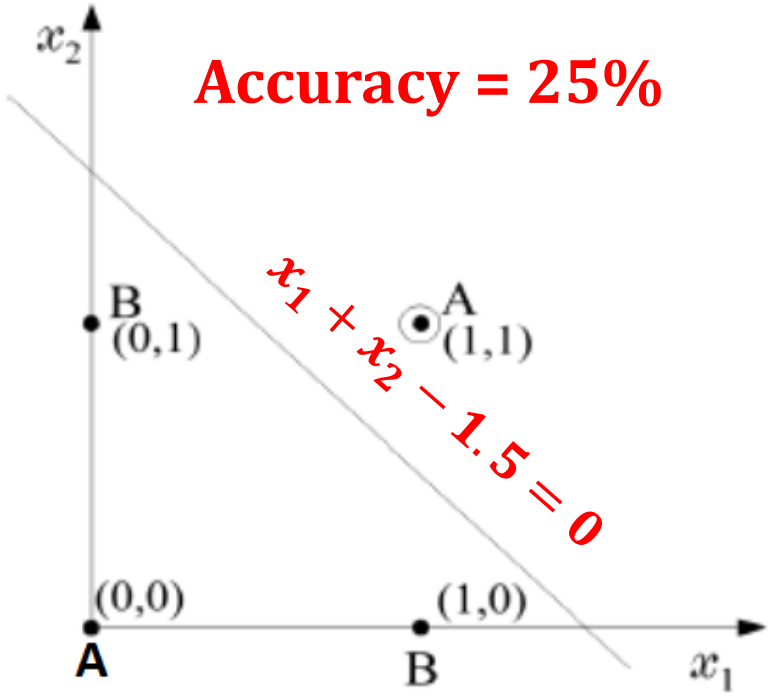
x_1	x_2	XOR	Class
0	0	0	B
0	1	1	A
1	0	1	A
1	1	0	B

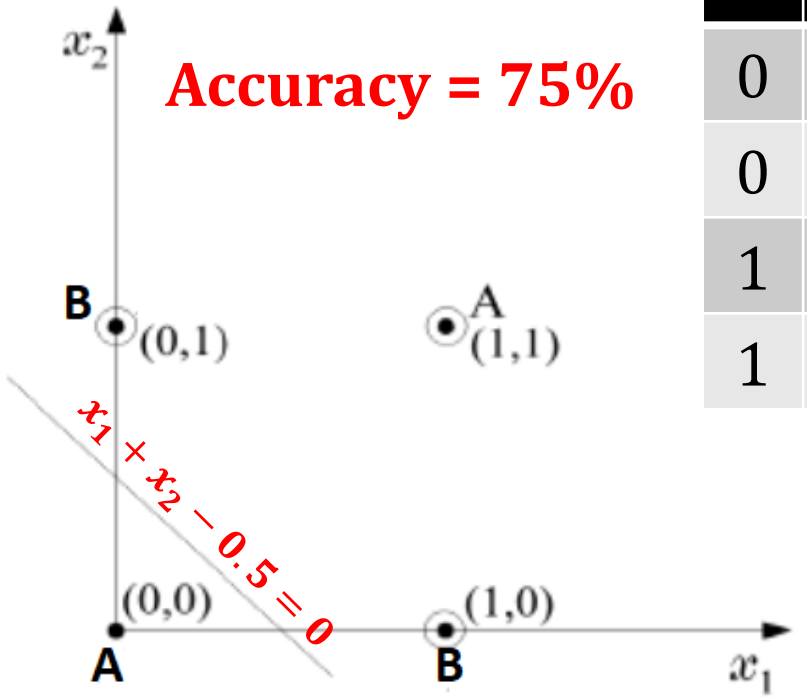


Example (XOR)

- There is no single line (hyperplane) that separates class A from class B. On the contrary, AND and OR operations are linearly separable problems

x_1	x_2	o/p	tar get
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

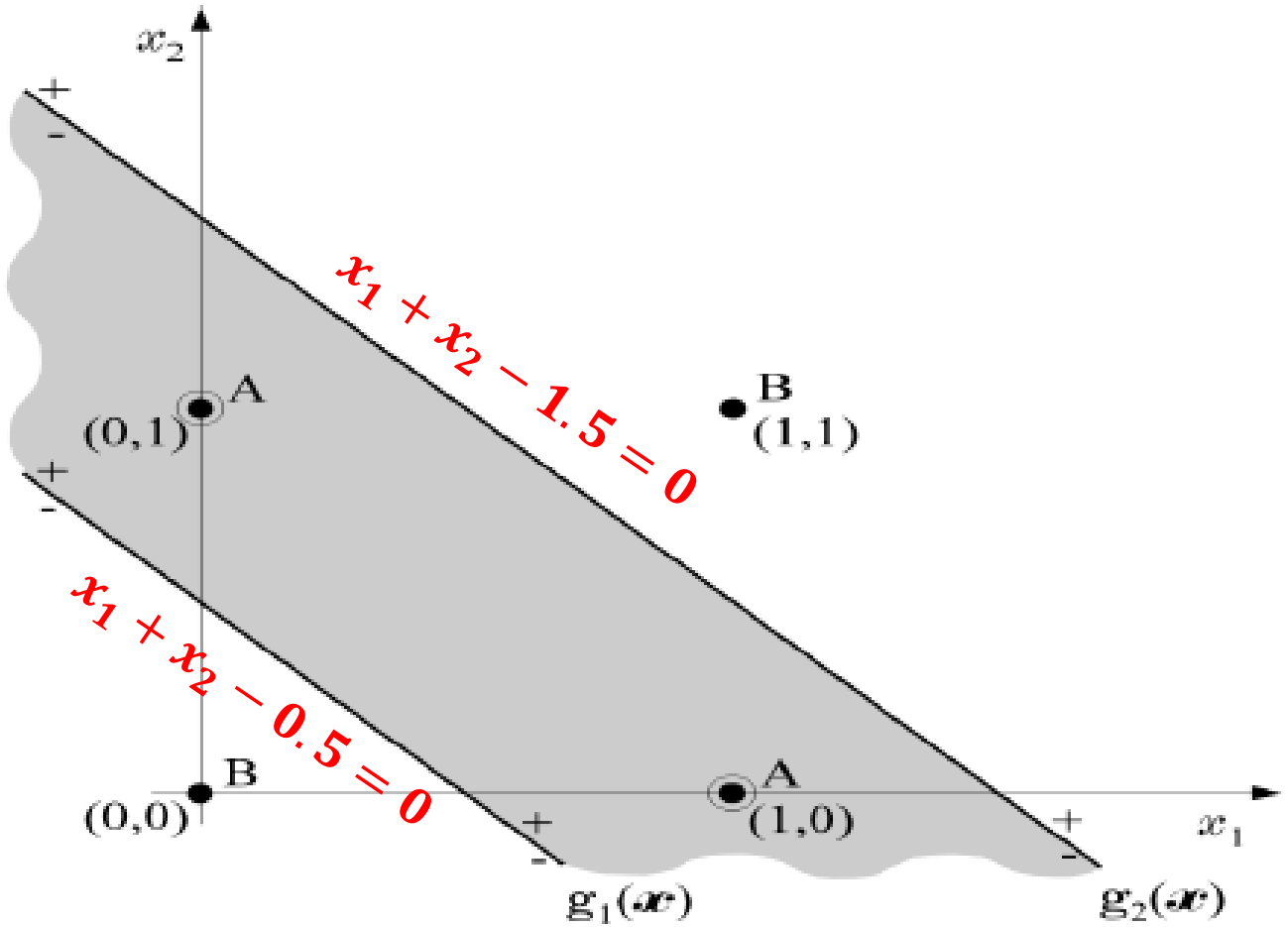




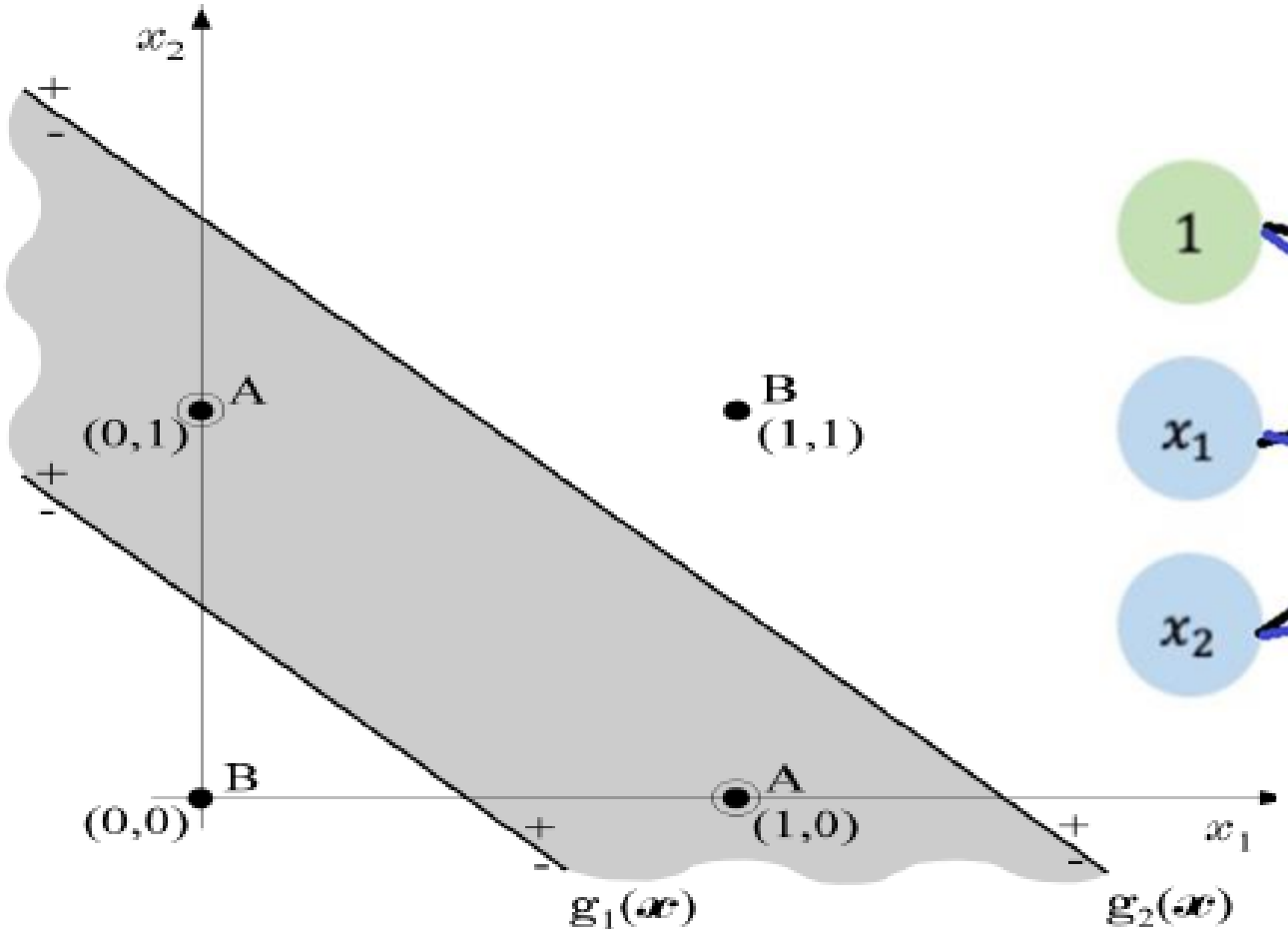
x_1	x_2	o/p	tar get
0	0	0	0
0	1	1	1
1	0	1	1
1	1	1	0

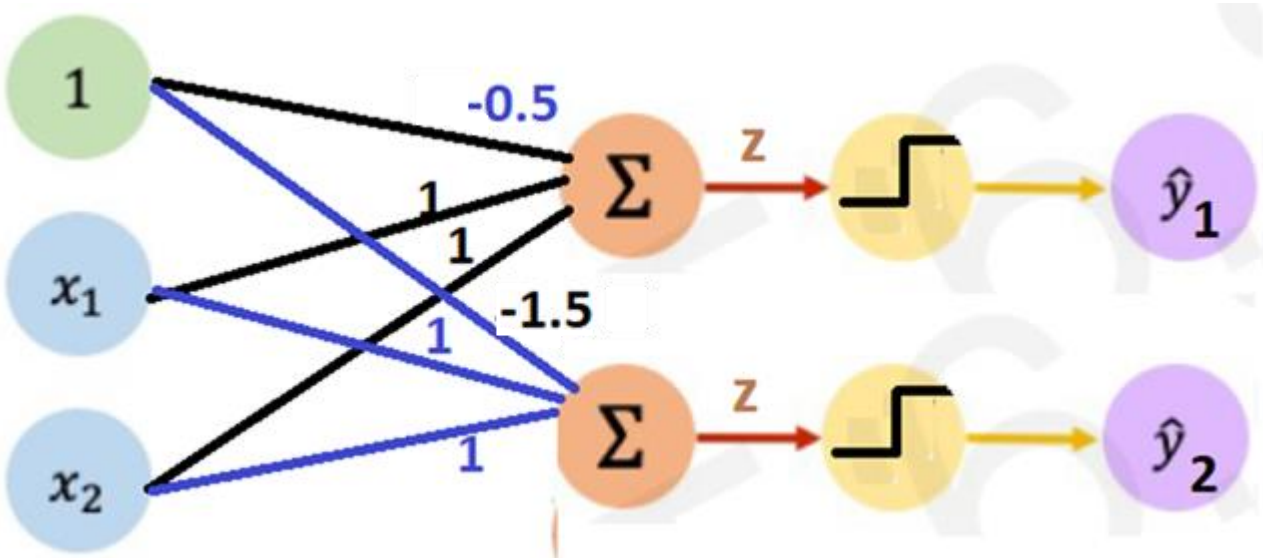
Example (XOR)

- For the XOR problem, draw two, instead, of one lines



Example (XOR)





Two-layer NN (example)

1st phase				2nd phase
x_1	x_2	y_1	y_2	
0	0	0(-)	0(-)	(0)
0	1	1(+)	0(-)	(1)
1	0	1(+)	0(-)	(1)
1	1	1(+)	1(+)	(0)

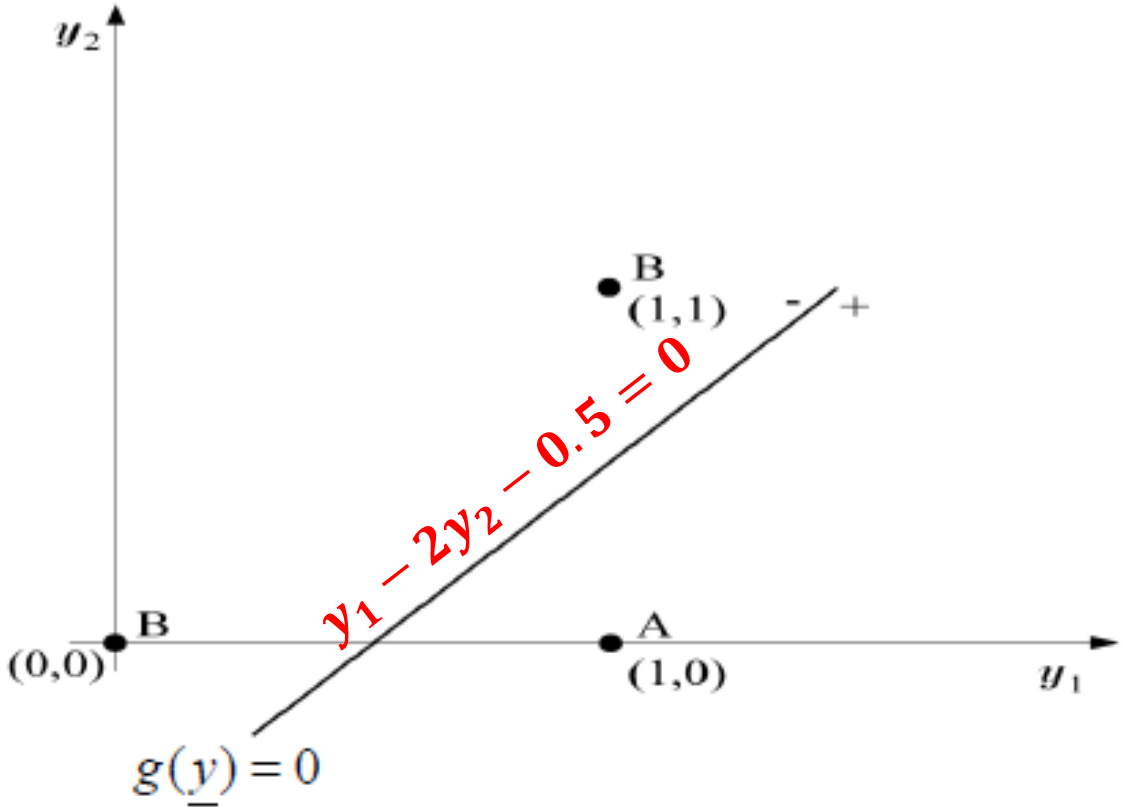
- Equivalently: The computations of the first phase perform a mapping

$$\underline{x} \rightarrow \underline{y} = [y_1, y_2]^T$$

Two-layer NN (example)

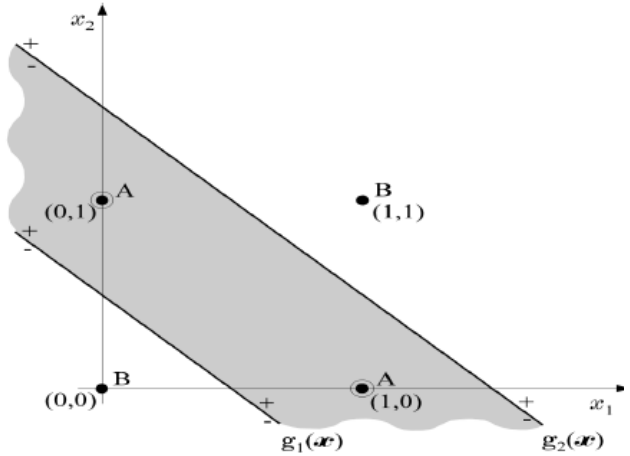
The decision is now performed on the transformed $\underline{y}$ data.

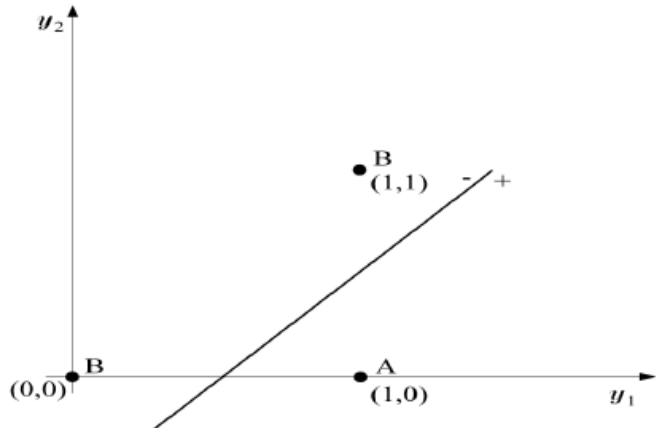
y_1	y_2	2nd phase
0(-)	0(-)	B(0)
1(+)	0(-)	A(1)
1(+)	0(-)	A(1)
1(+)	1(+)	B(0)

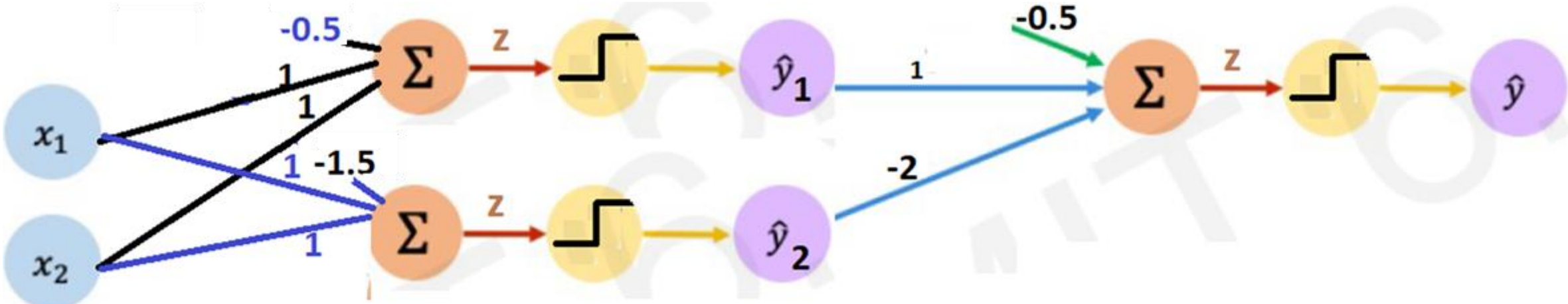


This can be performed via a second line, which can also be realized by a perceptron.

Two-layer NN (example)







This is known as the two layer perceptron with one hidden and one output layer.

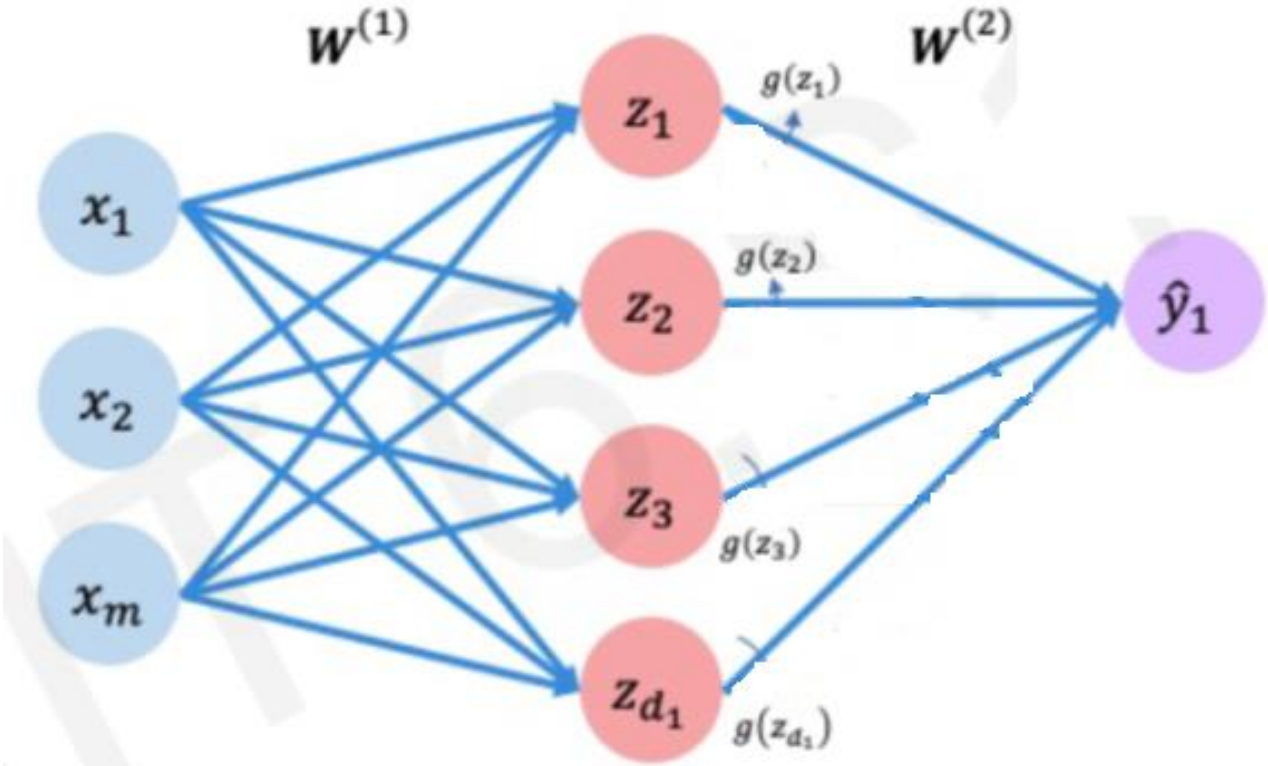
Two-layer NN (example)

x_1	x_2	$\hat{y}_1$	$\hat{y}_2$	$\hat{y}$	target
0	0	0	0	0	0
0	1	1	0	1	1
1	0	1	0	1	1
1	1	1	1	0	0

1st phase				2nd phase
x_1	x_2	y_1	y_2	
0	0	0(-)	0(-)	B(0)
0	1	1(+)	0(-)	A(1)
1	0	1(+)	0(-)	A(1)
1	1	1(+)	1(+)	B(0)

Accuracy=100%

Two-layer NN

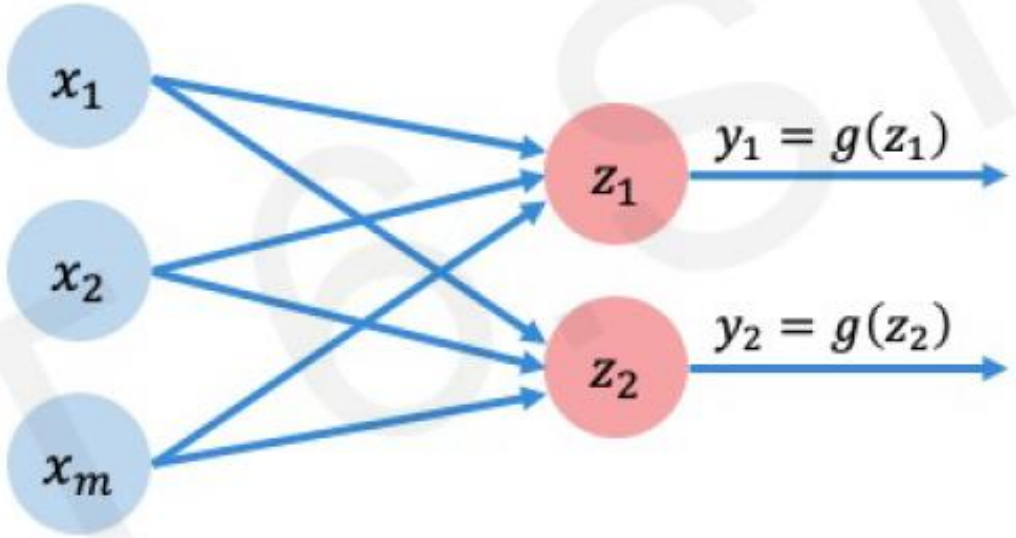


$$z_i = w_{0,i}^{(1)} + \sum_{j=1}^{m} x_j \, w_{j,i}^{(1)}$$

$$\hat{y}_i = g\left(w_{0,i}^{(2)} + \sum_{j=1}^{d_1} g(z_j) \, w_{j,i}^{(2)} \right)$$

Multi-output perceptron

Because all inputs are densely connected to all outputs, these layers are called **Dense** layers

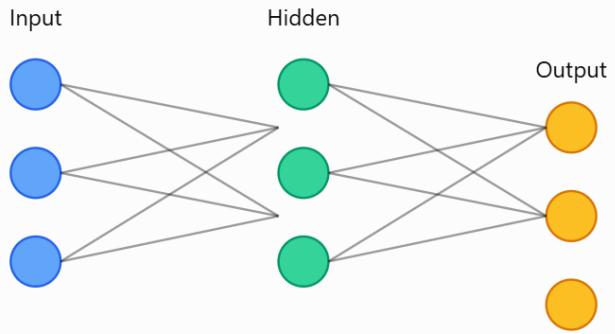


$$z_i = w_{0,i} + \sum_{j=1}^{m} x_j \, w_{j,i}$$

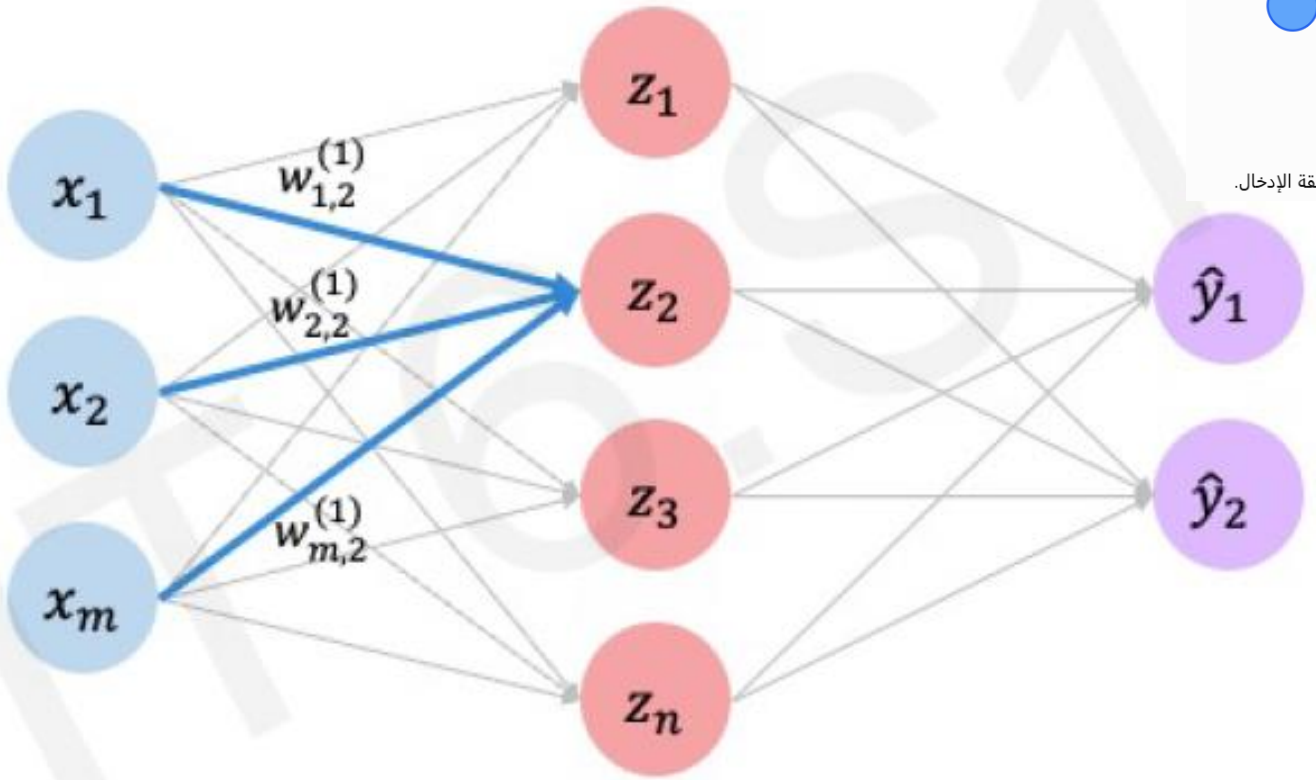
Two-layer NN

1. Two-layer Neural Network

شبكة عصبية ذات طبقتين، حسب طريقة العد الشائعة في هذا السياق.



- طبقة مخفية واحدة (Hidden Layer).
- طبقة إخراج واحدة (Output Layer).
- في طريقة العد دي، بنحسب الطبقات التي تحتوي على أوزان، ولا نحسب طبقة الإدخال.



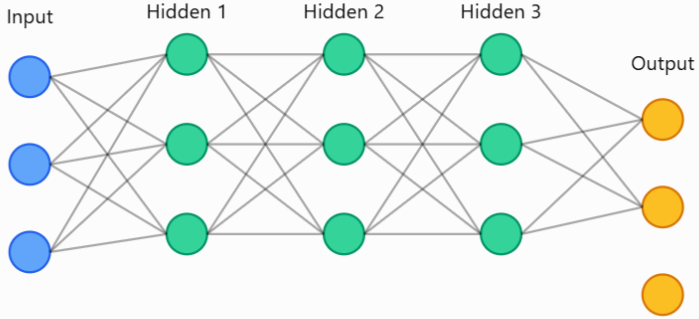
$$z_2 = w_{0,2}^{(1)} + \sum_{j=1}^{m} x_j \, w_{j,2}^{(1)}$$

$$= w_{0,2}^{(1)} + x_1 \, w_{1,2}^{(1)} + x_2 \, w_{2,2}^{(1)} + x_m \, w_{m,2}^{(1)}$$

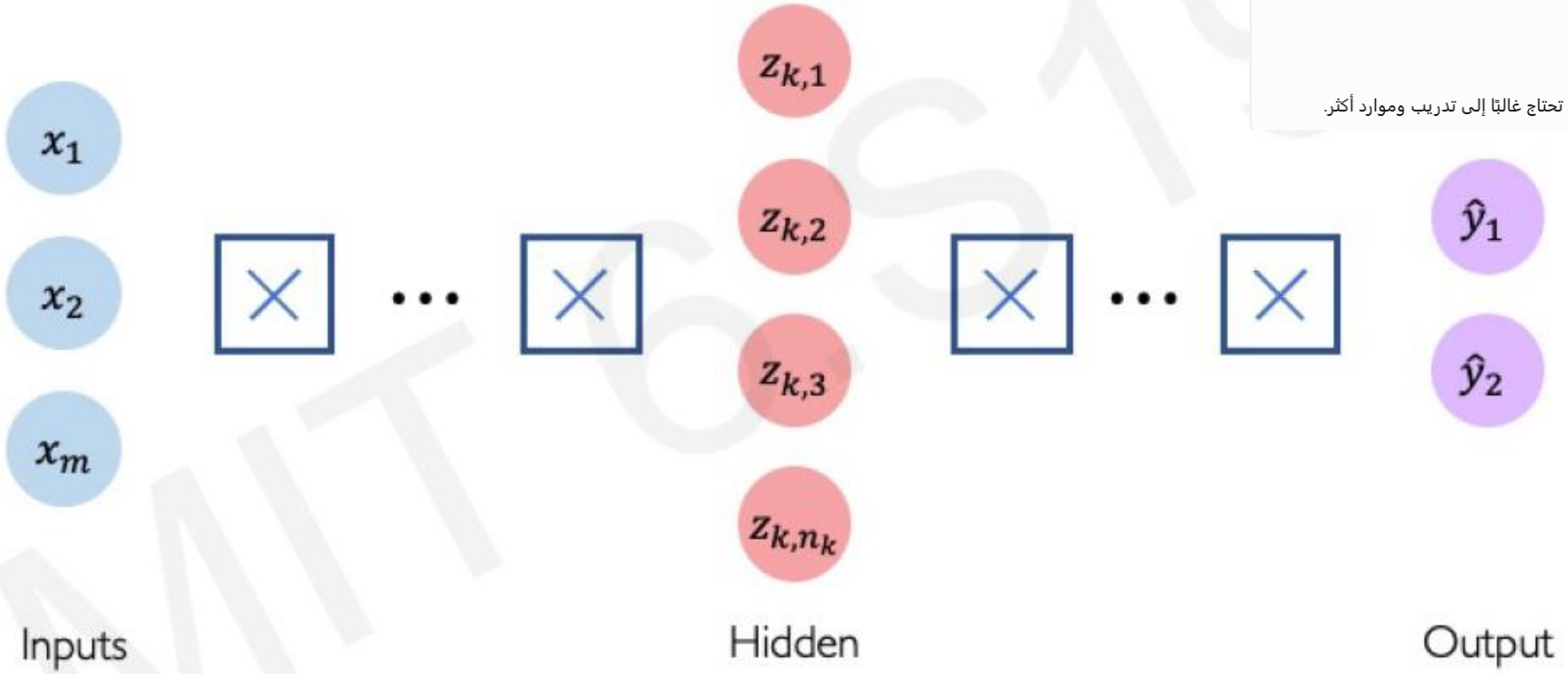
DNN: Multi-layer perceptron (MLP)

2. DNN — Deep Neural Network (مثل MLP عميقة)

شبكة فيها عدة طبقات مخفية.



- تحتوي على عدة Hidden Layers.
- كلما زاد عدد الطبقات المخفية، زاد عمق الشبكة.
- يمكن أن تتعلم أنماطًا وعلاقات أكثر تعقيدًا، لكنها تحتاج غالبًا إلى تدريب وموارد أكثر.



ملخص الفرق

وجه المقارنة	Two-layer NN	DNN
عدد الطبقات المخفية	واحدة غالبًا	عدة طبقات
العمق	أقل	أكبر
التعقيد الذي يمكن تعلمه	أبسط نسبيًا	أكثر تعقيدًا غالبًا
التدريب	أبسط غالبًا	قد يكون أصعب

$$z_{k,i} = w_{0,i}^{(k)} + \sum_{j=1}^{n_{k-1}} g(z_{k-1,j})\, w_{j,i}^{(k)}$$

Thank You

